

The Java Environment (AWS, n.d.a.)

- **Definition:** Free-to-use, highly dynamic, high-level programming language used for coding web applications, mobile applications, enterprise software, big data applications, and server-side technologies.
- **Origin:** Created by Sun Microsystems in 1995 as a multi-platform, object-oriented, network-centric language that can serve as a platform itself.
- **Common Uses:** Game development, cloud computing, big data, artificial intelligence (AI), and Internet of Things (IoT).

Java Runtime Environment (JRE)

- **Definition:** The software required for Java programs to run correctly, providing necessary class libraries and resources.

Advantages of Using Java

- **Active Community Support:** Popularity and longevity provide an active community to support developers.
- **High-Quality Learning Resources:** Detailed documentation, books, and courses offer support to new programmers.
- **Inbuilt Functions and Libraries:** Rich ecosystem of pre-built functions and libraries eliminates the need to code everything from scratch.
- **Security:** Configurable security levels run untrusted code in a secure environment, preventing viruses or unauthorized file access.
- **Platform Independent:** "Write Once, Run Anywhere" (WORA) philosophy allows code to run across Windows, iOS, Android, and Linux without rewriting.

Core Understanding Requirements To use Java, developers must understand:

1. Java Virtual Machine (JVM)
2. Java Language and APIs

Java Virtual Machine (JVM)

- **Definition:** A virtual machine serving as a run-time engine to execute Java applications.
- **Language Translation Categories:**
 - **Compilers:** Translates natural, English-like source code entirely into machine language/bytecode binary.
 - **Interpreters:** Reads bytecode and interacts with the operating system to execute instructions line by line.
- **Hybrid Approach:** Java was the first language to combine both techniques using the JVM, translating intermediate Java bytecode into machine language for execution.
- **File Processing:** Source code is saved as a `.java` file. The compiler converts it to bytecode, which the Java interpreter (`java.exe`) executes line-by-line within the JVM.

Java APIs

- **Definition:** Application Programming Interfaces (APIs) are essential software components bundled directly with the Java Platform. They consist of pre-written Java programs that allow developers to integrate existing functionality into their code (e.g., fetching date and time, running mathematical operations, or manipulating text strings).
- **Language vs. API Scope:** The core Java language relies on roughly 50 keywords, whereas the Java API contains several thousand classes and thousands of methods available for immediate use.

Java Skeleton (Soma, T. & Streib, J., 2023)

- **Overview:** Examining and dissecting a simple program is the most effective approach to understanding the syntax and structural foundations of the language.

```
1 class Skeleton {  
2     public static void main (String[] args){  
3         System.out.println("First Java Application"); //for display  
4     }  
5 }
```

Here is the structured breakdown of the next section, continuing in exact order:

Line 1 Breakdown: Classes and Identifiers

- **Class Definition:** The line starts with the keyword `class`, which serves as the foundational unit of any Java program. It defines a group of objects containing:
 - **Data Members:** Variables that store data.
 - **Methods:** Functions containing program logic.
 - *Analogy:* A class is the blueprint for a house, while the house itself is the object.
- **Class Name:** The word `Skeleton` on Line 1 is the user-defined name for the class.

Identifiers

- **Definition:** An identifier is a name assigned to a program component (e.g., class, object, or variable).
- **Naming Rules for Identifiers:**
 - Must start with an English letter, an underscore (`_`), or a dollar sign (`$`). It **cannot** start with a digit.
 - Can contain any combination of letters, digits, underscores, or dollar signs, but **no white spaces**.
 - Is **case-sensitive** (e.g., `NUMBER`, `number`, and `Number` are three distinct identifiers).
 - Cannot use reserved primitive values: `true`, `false`, or `null`.
 - Cannot be a Java reserved word.

Reserved Words

- **Definition:** Keywords with predefined significance to the Java compiler. They cannot be used as identifiers or variable names (names referring to data stored in memory).

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Line-by-Line Code Breakdown

- **Braces ({ and }):** Used to identify blocks of code and data. Every opening brace must have a matching closing brace. The entire **Skeleton** class is enclosed between the opening brace on line 1 and the closing brace on line 5.
- **Method Modifiers (public static void):**
 - **public:** Access/visibility modifier that allows the Java interpreter to find and execute the **main()** method.
 - **static:** Defines the method as a class method, allowing it to execute without creating an object of the class.
 - **void:** Indicates that **main()** is a non-value-returning method.
- **main Method:** The entry point of any Java application. Program execution must always begin with **main()**. Every Java application must contain at least one **main()** method.
- **Parameters (String args[]):**
 - **Parameters:** Listed inside parentheses after the method name to receive incoming values.
 - **args:** Name of the parameter array.
 - **String:** A sequence of characters (letters, numbers, symbols).
 - **Square Brackets ([]):** Indicate that **args** is an array (a collection of items of the same data type).
- **Statements & Literals (Line 3):**
 - **Statement:** An executable instruction telling the compiler what action to perform.
 - **Literal String:** A sequence of characters enclosed in double quotation marks ("...") that displays exactly as written.
 - **Arguments:** Information passed into a method so it can perform its task.
- **Comments:** Explanations added to make code easier for programmers to read; they are ignored during compilation and execution.
 - **Single-Line Comment (//):** Starts with **//** and extends to the end of the line.
 - **Multi-Line Comment (/* ... */):** Enclosed between **/*** and ***/** for comments spanning multiple lines.

Saving, Compiling, and Executing a Program (Agarwal & Bansal, 2023)

- **Java Development Kit (JDK):** Software package that includes the JRE along with tools for developing, debugging, and monitoring Java applications. Must be installed to write, compile, and run Java programs.

1. Write the Source Code:

Type the following Java code into a text editor like Notepad:

```
Java
public class Demo {
    //program begins from here starts with
    public static void main (String a []) {
        System.out.println("First program in Java");
    }
}
```

2. Save the File Correctly:

Prevents automatic .txt extension.

Save the file as "**Demo.java**" using double quotation marks in Notepad so it does not append .txt to the filename.

3. Navigate to the File Directory:

Open Command Prompt (cmd) and use the **cd** command to change to the directory where **Demo.java** is saved.

4. Set the JDK Path:

Configure the operating system path so it can find the Java compiler:

```
set path=%path%;[JDK bin directory]
```

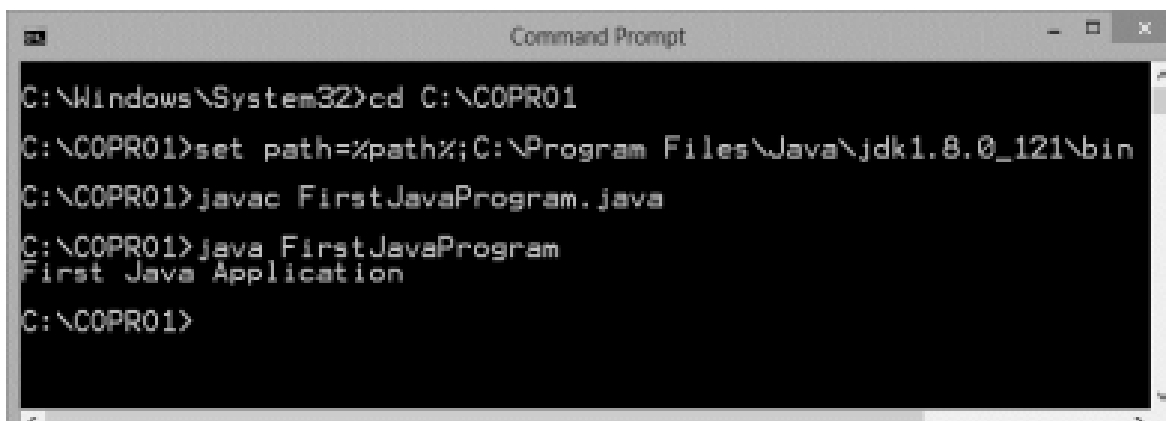
5. Compile the Java Program:

Translate the source code into bytecode using the Java compiler (**javac**):

```
javac Demo.java
```

6. Execute the Compiled Program:

Run the compiled bytecode using the Java interpreter (**java**):



```
Command Prompt

C:\Windows\System32>cd C:\COPR01
C:\COPR01>set path=%path%;C:\Program Files\Java\jdk1.8.0_121\bin
C:\COPR01>javac FirstJavaProgram.java
C:\COPR01>java FirstJavaProgram
First Java Application
C:\COPR01>
```

